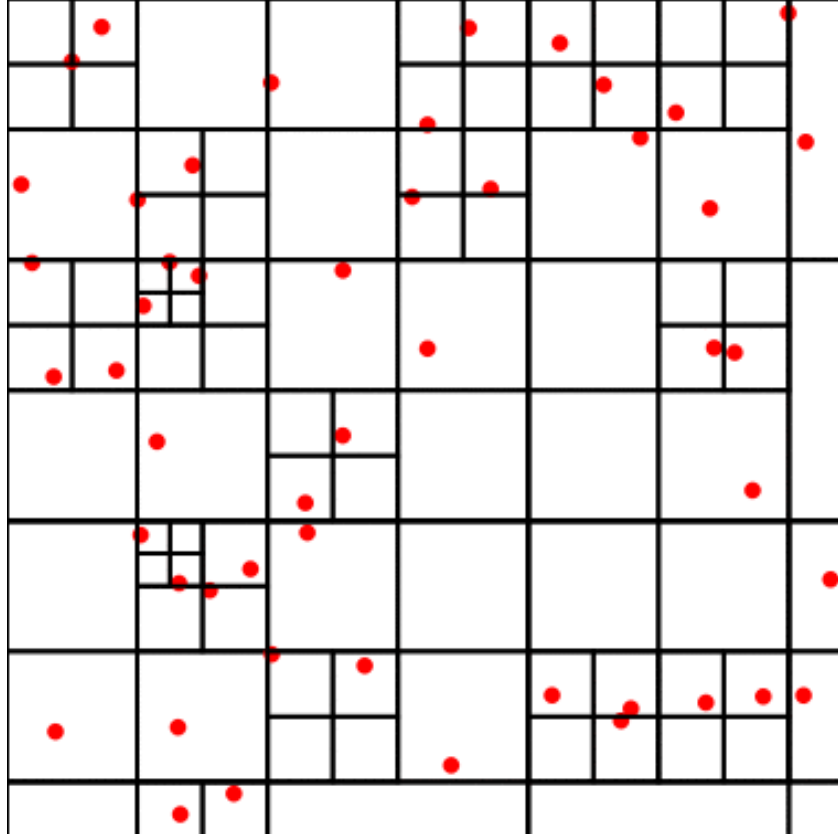




Parallel Computing Barnes Hut

Dr. Torsten Bosse
Advanced Computing, FSU Jena

n-Body Problem



Simulate n-bodies in a two-dimensional domain over a certain time period.

Each body/particle “i” is described by its

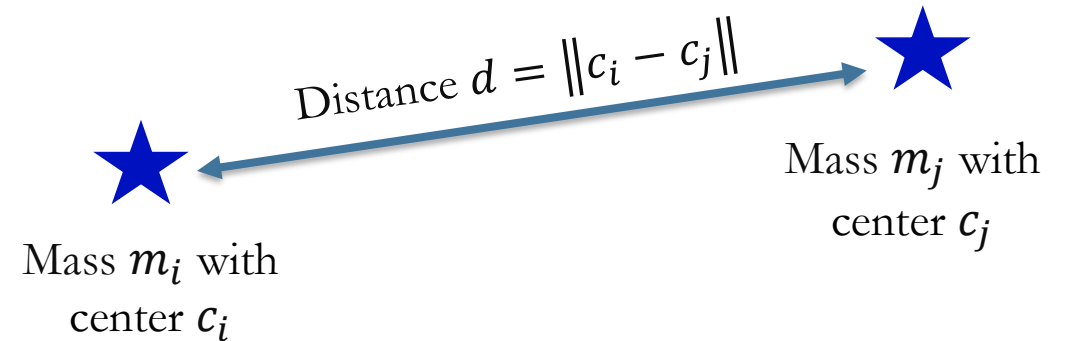
- mass m_i ,
- positional coordinates $c_i = (x_i, y_i)$,
- directional velocities $v_i = (v_i^x, v_i^y)$.

2-Body Problem

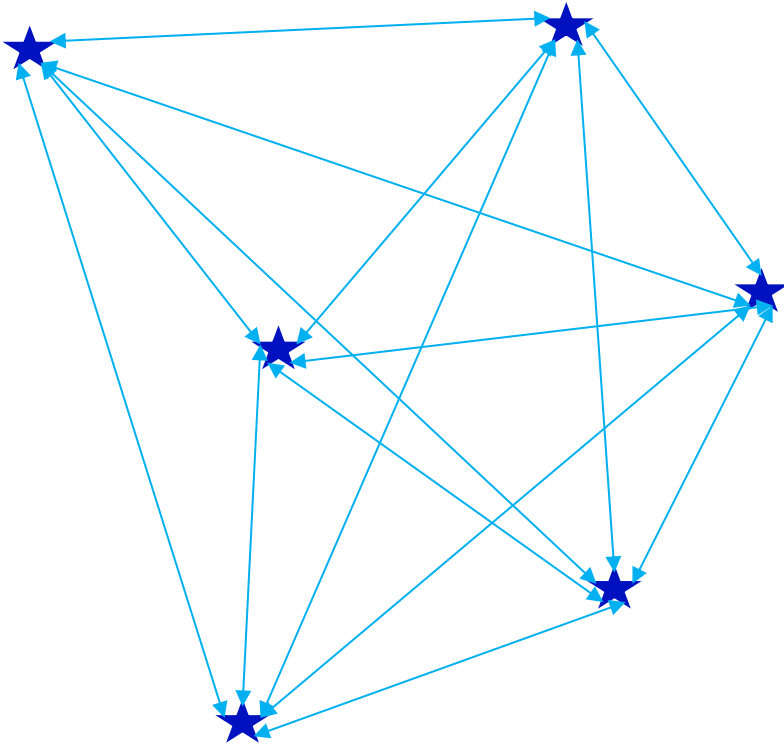
Gravitation between two bodies cause an acceleration of the bodies and, thus, a change in their velocity and position over time t .

Gravitational force depends

- on the Gravitational constant G ,
- on the two masses m_i and m_j ,
- as well as the distance $d = \|c_i - c_j\|$ between two bodies.



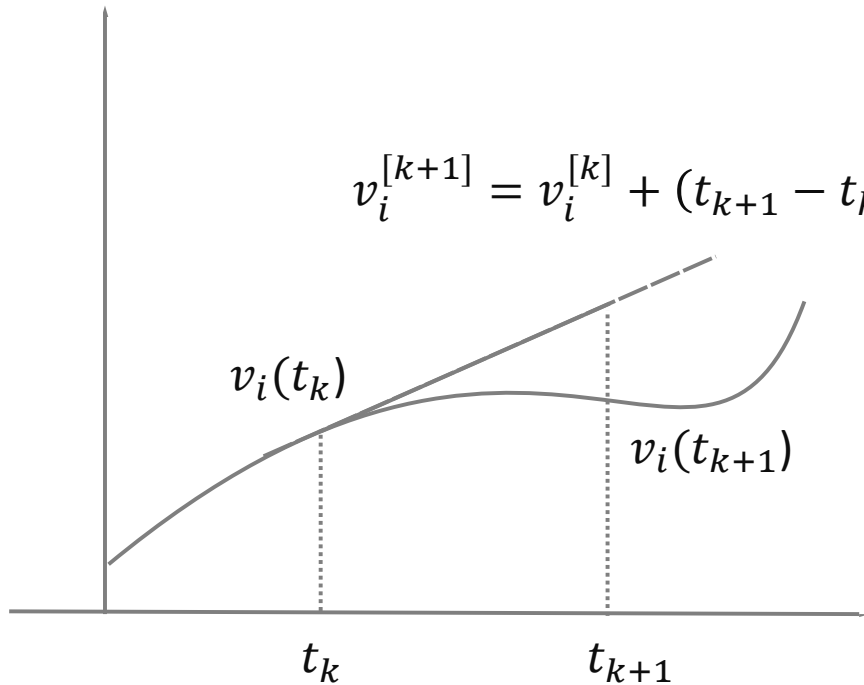
n-Body Problem



Idea of 2-body problem can be extended to the n-body problem, where the change in the velocity w.r.t. time t is described by the term

$$\frac{dv_i}{dt} = \sum_{i \neq j} \frac{Gm_j(c_i - c_j)}{\|c_i - c_j\|^3}$$

Explicit Euler



$$v_i^{[k+1]} = v_i^{[k]} + (t_{k+1} - t_k) \frac{dv_i(t)}{dt}(t_k)$$

Starting at time t_0 , explicit Euler method computes at each time t_k in the equidistant time discretization

$$t_k = t_0 + \varepsilon k, k = 0, 1, 2, \dots$$

the velocity for the next time step by

$$v_i^{[k+1]} = v_i^{[k]} + \varepsilon \sum_{i \neq j} \frac{G m_j (c_i - c_j)}{\|c_i - c_j\|^3}$$

and coordinates

$$c_i^{[k+1]} = c_i^{[k]} + \varepsilon v_i^{[k]}$$

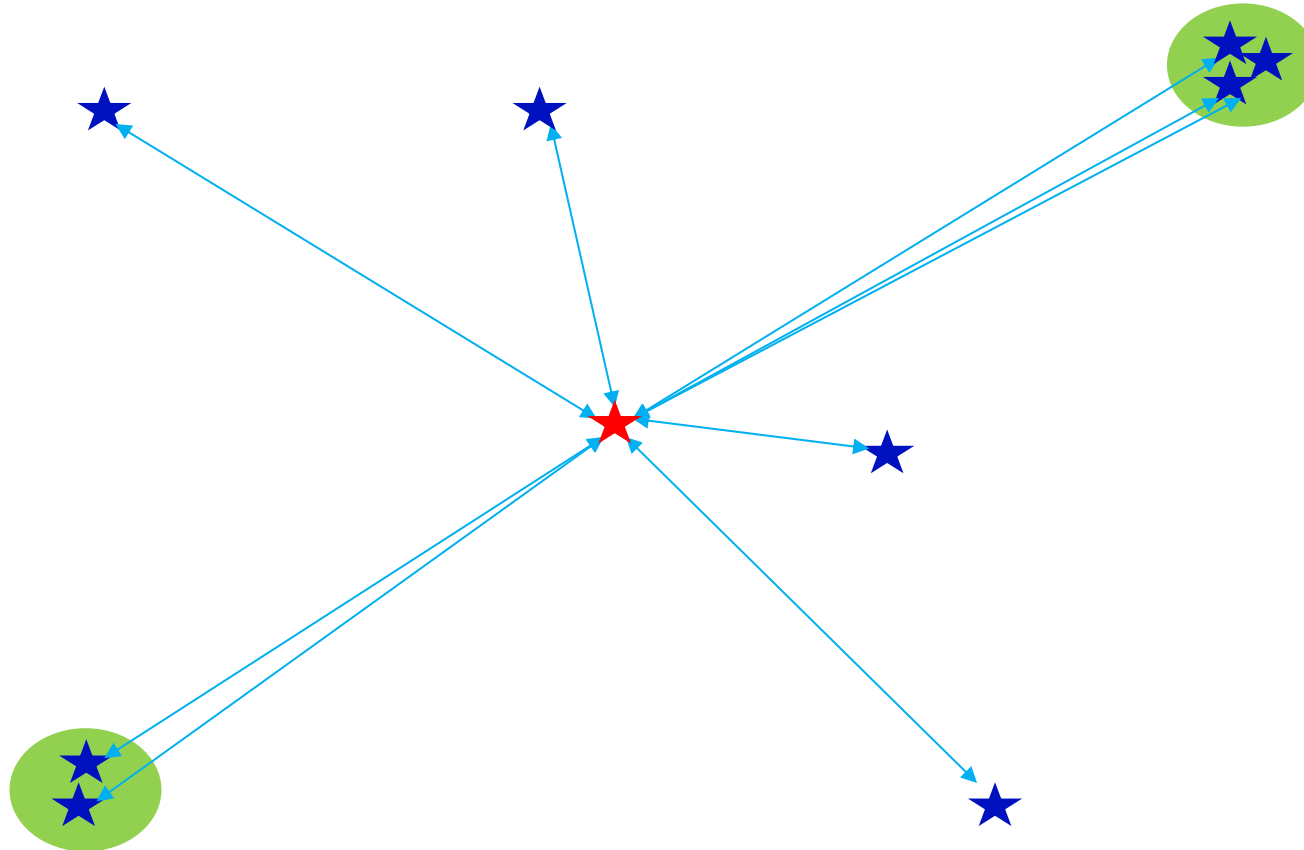
n-Body Simulation

```
void main (){
    \\Initialize particles,...
    ...
    \\Iterate over the time
    for (k=0; k < nt; k++){
        // Compute for each mass the coordinate and velocity update
        for (i=0; i < n; i++){
            // Compute for each mass the forces
            for (j=0; j < n; j++){
                if(i!=j){
                    // Update velocity by considering the gravitational forces
                    // exhibited from all other masses j!=i
                }
            }
            // Update coordinates
        }
        //Output current coordinates and velocities
    }
}
```

n-body Simulation requires $\Theta(n^2)$ operations per time step !!!

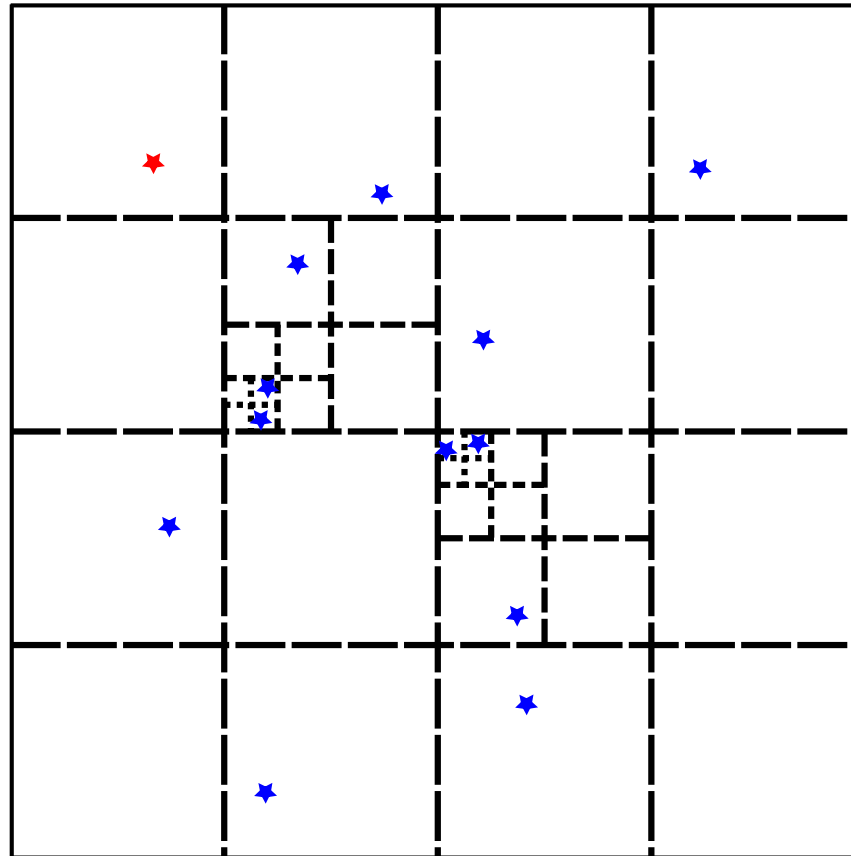
Clustering

Instead of computing all **pairwise forces**, cluster particles that are close to each other but far away from the **current particle** and treat the cluster of particles as one particle.



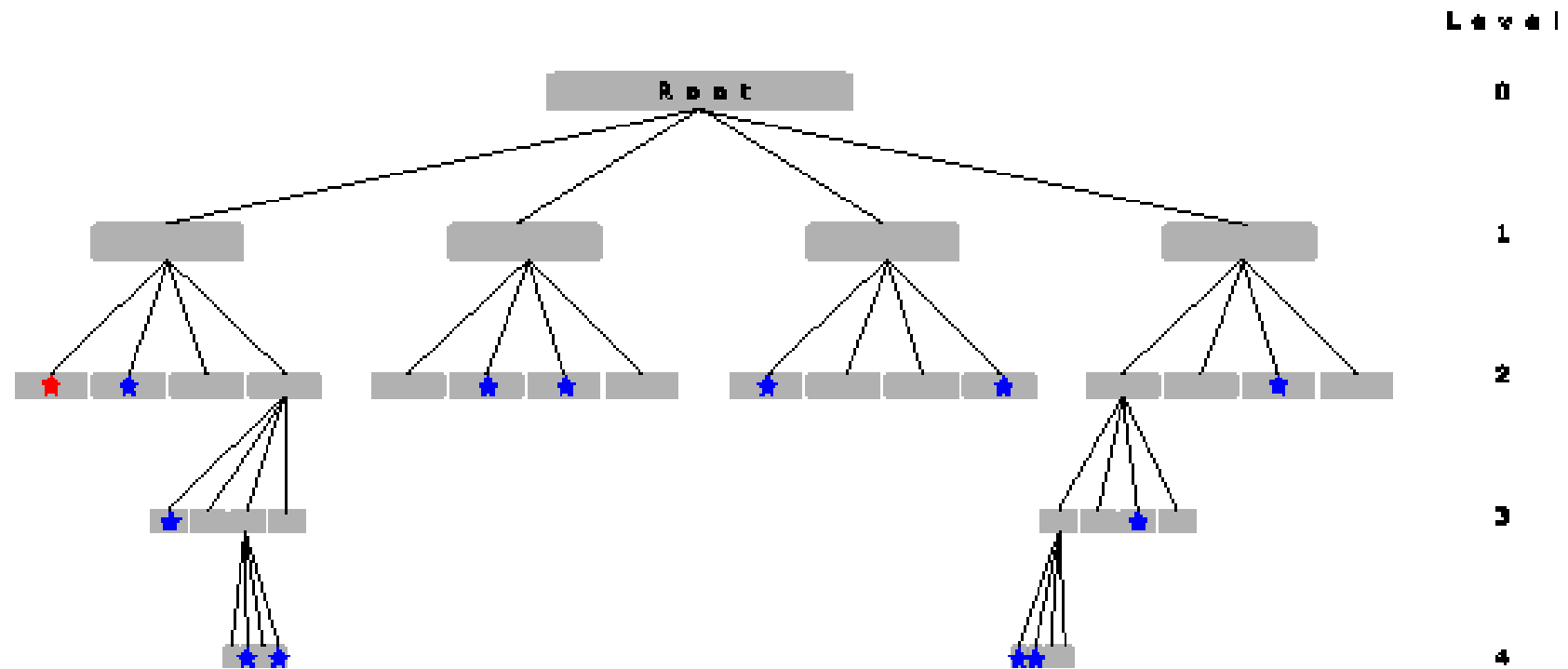
Domain Partitioning

Recursively partition the (rectengular) domain into four equidistant subdomains such that each subdomain has at most one particle.

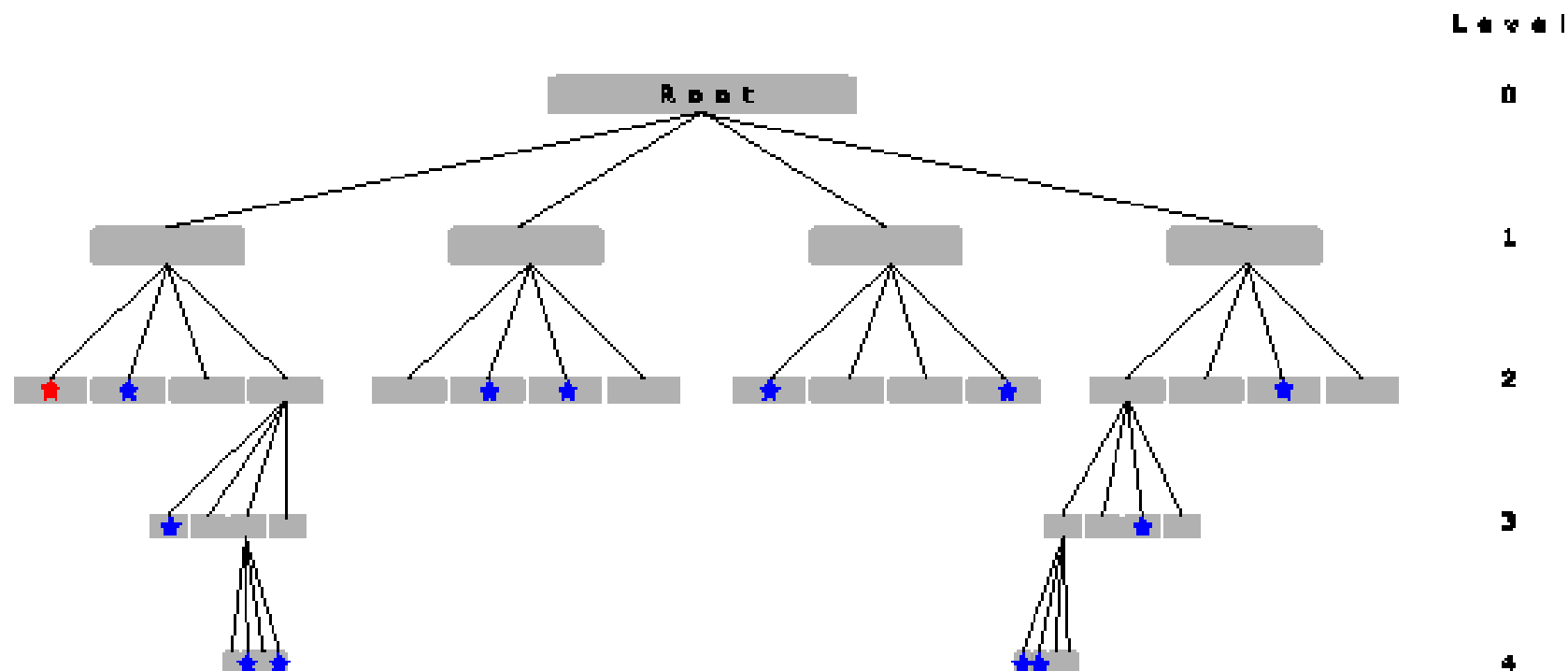
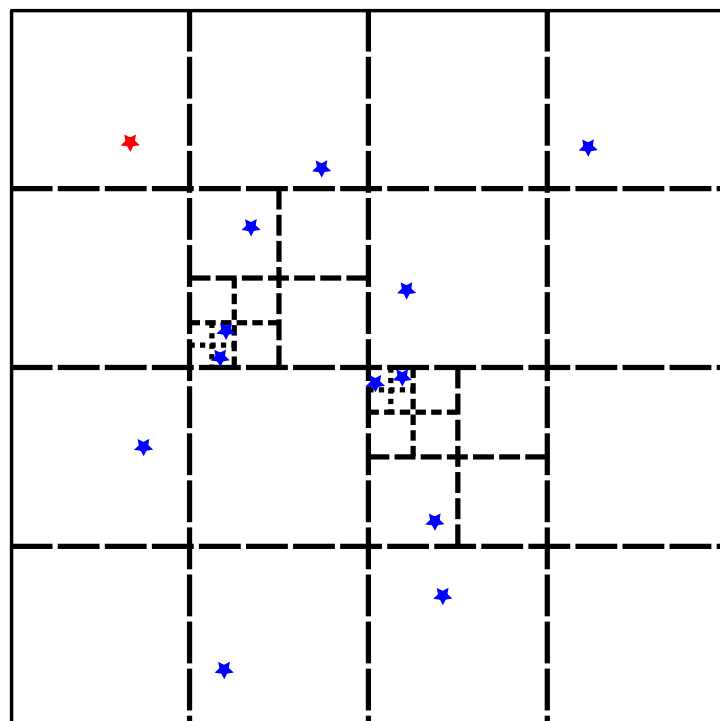


Quad-Tree structure

Recursive partitioning can be represented by quad-tree, namely, a tree whose nodes are either leaf nodes or subtree themselves.

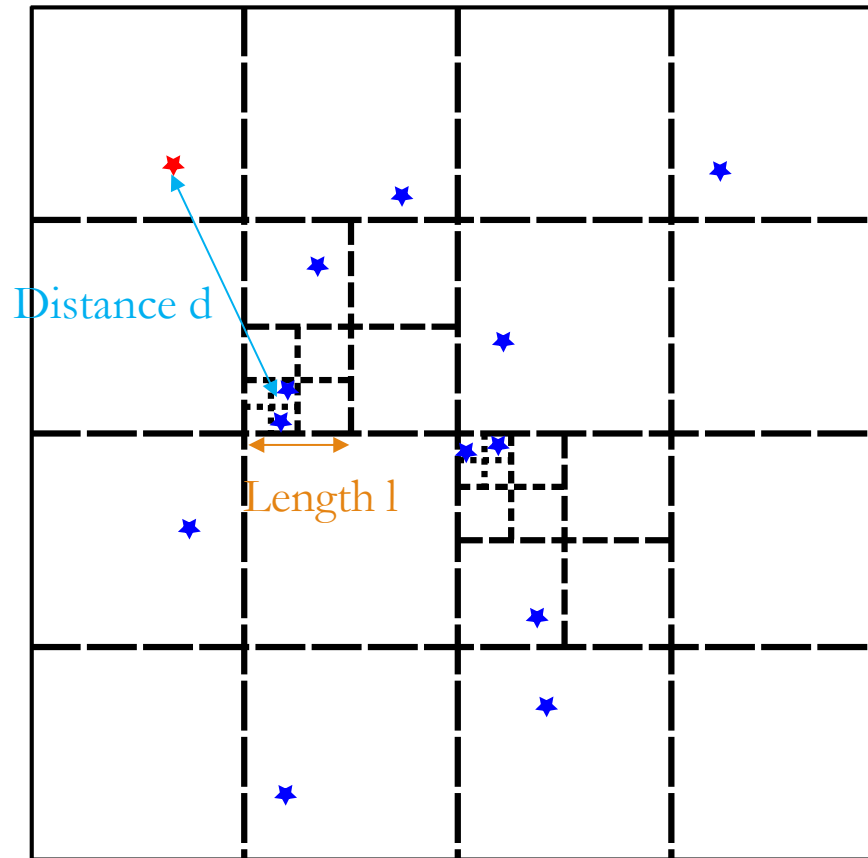


Domains as Quad-Trees



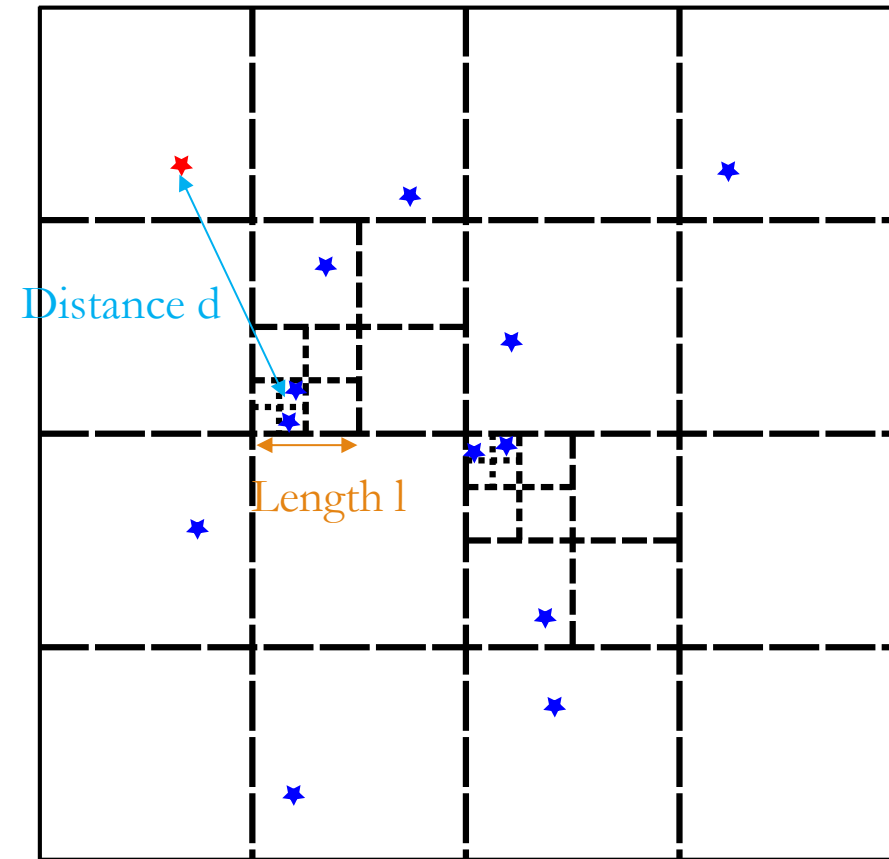
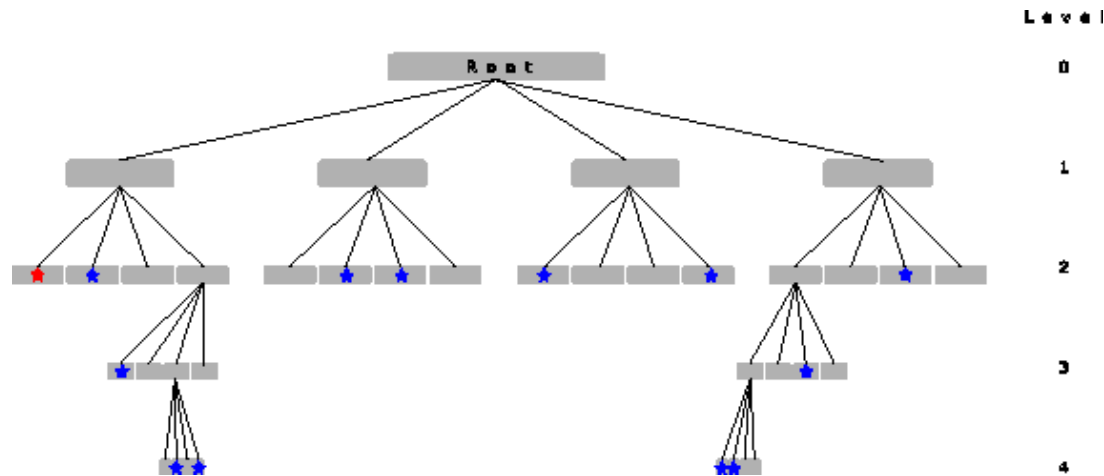
Clustering condition

Points in a domain are considered as clusters if $\frac{l}{d} \leq \theta$ for given value θ . Here, l is the size of the domain and d is the distance of the current particle to the mass center of the domain.



n-Body Simulation using Barnes-Hut Algorithm

```
void main (){
    \\Initialize particles,...
    ...
    \\Iterate over the time
    for (k=0; k < nt; k++){
        // Build quadtree with domain masses and their centers
        // Update velocity by recursively traversing the tree
        // Update coordinates
    }
    //Output current coordinates and velocities
}
```



Memory Access Pattern using Barnes-Hut Algorithm

